



MIDDLE SCHOOL COMPUTER SCIENCE

# Python-B with Artificial Intelligence

A 60-hour curriculum integrating Python programming, machine learning, language models, classification, recommendation systems, and responsible AI.

| 9 units            | 30 meetings | 60 academic hours |
|--------------------|-------------|-------------------|
| Python foundations | AI projects | Ethics throughout |

English working edition based on the Israel Ministry of Education program 'AI2 with Algorithmics through Python - Part B'. Target band: middle school, officially framed for Grade 8 and adaptable for Grade 9.

# Program at a Glance

Purpose, scope, structure, and progression

The program treats artificial intelligence as a recurring application context for computer science, not as a stand-alone topic. Each major Python concept becomes a tool for examining or building an AI-related system. Students move from deterministic algorithms to data-driven systems while maintaining a strong focus on testing, evidence, and human responsibility.

|                                                                                                |                                                                                         |                                                                                                   |
|------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| <b>Audience</b><br>Middle-school learners continuing from introductory Python.                 | <b>Duration</b><br>60 academic hours, commonly scheduled as 30 meetings of 90 minutes.  | <b>Approach</b><br>Problem solving, coding labs, inquiry, discussion, and project-based learning. |
| <b>Programming</b><br>Procedural Python with conditions, loops, functions, strings, and lists. | <b>AI</b><br>Machine learning, APIs, LLM concepts, classification, and recommendations. | <b>Responsibility</b><br>Bias, privacy, copyright, fairness, uncertainty, and human oversight.    |

## Curriculum map

| Unit | Title                                             | Hours    | Meetings   |
|------|---------------------------------------------------|----------|------------|
| 01   | Python-A Review: Conditions and Repetition        | 6 hours  | 3 meetings |
| 02   | Year-B AI Launch: From Rules to Learning          | 4 hours  | 2 meetings |
| 03   | Functions and Procedural Abstraction              | 6 hours  | 3 meetings |
| 04   | APIs: Bringing Intelligence into Code             | 4 hours  | 2 meetings |
| 05   | Strings and Text Processing                       | 8 hours  | 4 meetings |
| 06   | Advanced Language Models                          | 6 hours  | 3 meetings |
| 07   | Lists and Algorithmic Patterns                    | 10 hours | 5 meetings |
| 08   | Classification and Model Evaluation               | 8 hours  | 4 meetings |
| 09   | Recommendation Systems and Digital Responsibility | 8 hours  | 4 meetings |

## Scheduling note

The official master syllabus totals 60 hours. This working curriculum uses the practical/theory split shown in that master summary. A small number of detailed chapter tables in the source contain split totals that differ from the master summary; local programs should preserve the 60-hour total and may adjust the internal balance to fit their timetable.



# Learning Design and Year-Long Outcomes

What students should know, do, and value by the end of the program

## End-of-program outcomes

|                                                                                                                                                                                                                                                                                                                                                                                                                                  |                                                                                                                                                                                                                                                                                                                                                                                                   |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Students can build</b> <ul style="list-style-type: none"><li>- Modular Python programs using conditions, loops, functions, strings, and lists.</li><li>- Programs that validate input, handle expected failures, and test outputs.</li><li>- A controlled API-based application, a classifier, and a recommendation system.</li><li>- Evidence-based explanations of how data and design choices influence results.</li></ul> | <b>Students can reason</b> <ul style="list-style-type: none"><li>- When rules are sufficient and when learning from examples may be useful.</li><li>- How training data, test data, accuracy, diversity, and bias relate.</li><li>- Why language models can produce plausible but incorrect answers.</li><li>- How privacy, fairness, copyright, and human oversight affect deployment.</li></ul> |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## Recommended learning cycle

| Stage        | Teacher and student action                                            |
|--------------|-----------------------------------------------------------------------|
| 1. Encounter | Begin with a concrete problem, system, or surprising output.          |
| 2. Predict   | Students commit to a prediction before running code or a model.       |
| 3. Build     | Develop the Python concept through a structured implementation task.  |
| 4. Test      | Use normal, boundary, adversarial, and diverse cases.                 |
| 5. Explain   | Connect behavior to code, data, or model limitations.                 |
| 6. Improve   | Revise the algorithm, data plan, prompt, evaluation, or safeguards.   |
| 7. Reflect   | Identify what remains uncertain and who is responsible for decisions. |

## Cross-curricular habits

|                                                                               |                                                                                   |
|-------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| <b>Evidence</b><br>Record observations before making claims.                  | <b>Modularity</b><br>Name, test, and reuse small components.                      |
| <b>Uncertainty</b><br>Treat confidence and accuracy as measured, not assumed. | <b>Responsibility</b><br>Ask who benefits, who may be harmed, and who can appeal. |

# Python-A Review: Conditions and Repetition

6 hours | 3 meetings | 5 practical + 1 theory

Unit intent. Rebuild fluency with variables, Boolean logic, conditional execution, for loops, and while loops so every learner has a reliable foundation for the AI-integrated work that follows.

## Learning outcomes

- Construct simple and compound Boolean conditions using and, or, and not.
- Use if, elif, and else, including input validation and nested decisions.
- Select a counted for loop or condition-controlled while loop for a given problem.
- Trace, test, and explain the correctness and termination of loop-based programs.

### Python and CS focus

- Boolean expressions
- if / elif / else
- for with range
- while and termination

### AI focus

- Contrast deterministic output with probabilistic AI output
- Establish the habit of testing claims with examples

## Suggested meeting sequence

| Meeting | Core learning                     | Classroom product                            |
|---------|-----------------------------------|----------------------------------------------|
| 1       | Compound conditions and branching | Decision program with input validation       |
| 2       | Counted repetition                | Pattern, counter, and accumulator challenges |
| 3       | Conditional repetition            | Sentinel loop and debugging clinic           |

### Assessment evidence

Code tracing, short implementation tasks, and a mini-program that combines validation, branching, and repetition.

# Year-B AI Launch: From Rules to Learning

4 hours | 2 meetings | 2 practical + 2 theory

Unit intent. Introduce the central distinction between rule-based programming and systems that learn patterns from examples, while developing critical awareness of datasets, bias, fairness, and developer responsibility.

## Learning outcomes

- Explain the difference between explicit rules and machine learning from examples.
- Define dataset, pattern recognition, bias, and data diversity.
- Analyze how missing or unbalanced examples can change real-world decisions.
- Describe developer responsibilities before and after an AI system is deployed.

### Python and CS focus

- Rule-based logic
- Evidence chains
- Data collection plans
- Test cases

### AI focus

- Machine learning
- Datasets and bias
- Cultural and social bias
- Fairness and accountability

## Suggested meeting sequence

| Meeting | Core learning           | Classroom product                                 |
|---------|-------------------------|---------------------------------------------------|
| 1       | Rules versus examples   | Quick, Draw! experiment and dataset investigation |
| 2       | Bias and responsibility | Real-world case analysis and improvement plan     |

### Assessment evidence

Failure analysis, bias case explanations, dataset improvement plan, peer critique, and concept-summary statements.

# Functions and Procedural Abstraction

6 hours | 3 meetings | 4 practical + 2 theory

Unit intent. Use functions as reusable algorithmic building blocks and develop the distinction between what a component does and how it is implemented.

## Learning outcomes

- Define and call functions with and without parameters.
- Use return values to pass results back to the calling code.
- Decompose a problem into named, testable functions.
- Distinguish local variables from broader program state and avoid unnecessary globals.

### Python and CS focus

- def and calls
- Parameters
- return
- Scope
- Bottom-up design

### AI focus

- Prepare wrapper functions for later AI services
- Treat a model call as one component in a larger program

## Suggested meeting sequence

| Meeting | Core learning                    | Classroom product                          |
|---------|----------------------------------|--------------------------------------------|
| 1       | Functions without return values  | Refactor repeated code into procedures     |
| 2       | Parameters and reusable behavior | Parameter-driven drawing or game utilities |
| 3       | Returning values and scope       | Modular problem-solving challenge          |

### Assessment evidence

Trace function calls, implement specified interfaces, and submit a modular program with tests for each function.

# APIs: Bringing Intelligence into Code

4 hours | 2 meetings | 3 practical + 1 theory

Unit intent. Connect Python programs to an approved language-model service through an API while emphasizing secure key handling, dynamic prompts, efficient calls, validation, and resilient user experience.

## Learning outcomes

- Explain client-server communication and the role of an API.
- Write a wrapper function that sends a prompt and returns a response.
- Construct dynamic prompts with f-strings and validated user input.
- Recognize rate limits, cost, service errors, and safe secret management.

### Python and CS focus

- Imports
- Wrapper functions
- f-strings
- String accumulation
- Input validation

### AI focus

- API request-response cycle
- Prompt construction
- Rate limits
- Model uncertainty

## Suggested meeting sequence

| Meeting | Core learning                          | Classroom product           |
|---------|----------------------------------------|-----------------------------|
| 1       | API concepts and a secure wrapper      | First controlled model call |
| 2       | Dynamic prompts and robust interaction | Balanced Day mini-project   |

### Assessment evidence

Secure API use, a working return-based wrapper, one call outside the accumulation loop, and at least one resilience feature.

# Strings and Text Processing

8 hours | 4 meetings | 6 practical + 2 theory

Unit intent. Treat strings as structured data and apply indexing, slicing, methods, and traversal to practical text-processing problems that prepare learners for language-model concepts.

## Learning outcomes

- Use indexing and slicing with start, stop, and step.
- Apply core methods such as len, find, count, replace, upper, lower, startswith, and endswith.
- Validate alphabetic and numeric input and test membership with in.
- Traverse text by index or character and explain when each approach is useful.

### Python and CS focus

- Indexing
- Slicing
- String methods
- Traversal
- Validation

### AI focus

- Prompts as data
- Preprocessing and normalization
- Preparation for tokens and context

## Suggested meeting sequence

| Meeting | Core learning                 | Classroom product                |
|---------|-------------------------------|----------------------------------|
| 1       | String operators and indexing | Text transformation puzzles      |
| 2       | Methods and validation        | Input cleaner and analyzer       |
| 3       | Slicing                       | Pattern extraction and reversal  |
| 4       | Traversal                     | Mini text-processing application |

### Assessment evidence

Trace string operations, explain slicing, and build a tested text-processing program using methods and loops.

# Advanced Language Models

6 hours | 3 meetings | 4 practical + 2 theory

Unit intent. Build an age-appropriate mental model of tokenization, context windows, embeddings, semantic similarity, and self-attention, then apply the ideas in a Python semantic-guessing game.

## Learning outcomes

- Explain tokenization, token identifiers, and context-window limits.
- Describe embeddings as numerical positions in a multidimensional semantic space.
- Explain how context and self-attention change a word's effective meaning.
- Implement a semantic-similarity function and an interactive guessing loop.

### Python and CS focus

- External libraries
- Counters
- Percent conversion
- while loops
- Dynamic display

### AI focus

- Tokens
- Context windows
- Embeddings
- Self-attention
- Semantic distance

## Suggested meeting sequence

| Meeting | Core learning           | Classroom product                                  |
|---------|-------------------------|----------------------------------------------------|
| 1       | Tokens and tokenization | Tokenizer investigation across words and languages |
| 2       | Embeddings and context  | Semantic-map experiments                           |
| 3       | Semantic guessing game  | Build and demonstrate a Semantle-style project     |

### Assessment evidence

Tokenizer investigation log, completed similarity function, and a working game with accurate attempt counting and feedback display.

# Lists and Algorithmic Patterns

10 hours | 5 meetings | 7 practical + 3 theory

Unit intent. Use lists as dynamic ordered collections and combine list operations with traversal, accumulation, search, and procedural abstraction.

## Learning outcomes

- Create, access, update, and measure lists.
- Use append, insert, pop, remove, count, index, sort, reverse, split, and join appropriately.
- Implement accumulation and sequential-search patterns.
- Solve a larger problem with list-processing helper functions.

### Python and CS focus

- List operations
- Traversal
- Accumulation
- Sequential search
- Modular design

### AI focus

- Represent examples, ratings, and model results as collections
- Prepare data structures for recommendation systems

## Suggested meeting sequence

| Meeting | Core learning            | Classroom product                |
|---------|--------------------------|----------------------------------|
| 1       | Lists and indexing       | Collection manager               |
| 2       | Editing and list methods | Data-cleaning exercises          |
| 3       | Accumulation patterns    | Statistics from a list           |
| 4       | Sequential search        | Search with useful return values |
| 5       | Integrated list project  | Modular solution and test suite  |

### Assessment evidence

Code tracing, list-operation selection, search and accumulation implementations, and an integrated modular project.

# Classification and Model Evaluation

8 hours | 4 meetings | 4 practical + 4 theory

Unit intent. Understand classification, features, decision boundaries, training and test sets, accuracy, and visual bias through rule-based baselines and a Teachable Machine project connected back to Python.

## Learning outcomes

- Define classification, feature, and decision boundary.
- Distinguish useful features from irrelevant or misleading features.
- Plan separate training and test data and calculate accuracy.
- Identify background, lighting, pose, and angle biases and improve the data plan.

### Python and CS focus

- Complex conditions
- Classifier function
- Parallel lists
- Accuracy calculation
- Model loading

### AI focus

- Training and test sets
- Classification
- Decision boundaries
- Accuracy
- Visual bias

## Suggested meeting sequence

| Meeting | Core learning               | Classroom product                                  |
|---------|-----------------------------|----------------------------------------------------|
| 1       | Classification and features | Rule-based fruit or music classifier               |
| 2       | Training versus testing     | Exploratory Teachable Machine model                |
| 3       | Planned data collection     | 100+ examples per class with challenge cases       |
| 4       | Evaluation and integration  | External test set, accuracy, and Python connection |

### Assessment evidence

Rule-system edge-case analysis, feature justification, model plan, external testing, accuracy calculation, and bias-reduction proposal.

# Recommendation Systems and Digital Responsibility

8 hours | 4 meetings | 4 practical + 4 theory

Unit intent. Develop a user-based recommendation system in Python and critically examine content-based and collaborative filtering, filter bubbles, echo chambers, and the attention economy.

## Learning outcomes

- Compare content-based and user-based recommendation approaches.
- Calculate a similarity score from shared ratings and identify a digital twin.
- Recommend an unseen item liked by a similar user.
- Explain filter bubbles, echo chambers, and strategies for controlling a feed.

### Python and CS focus

- Parallel lists
- Nested logic
- Searching indices
- Functions
- CSV-style data

### AI focus

- Collaborative filtering
- Similarity
- Filter bubbles
- Echo chambers
- Attention economy

## Suggested meeting sequence

| Meeting | Core learning             | Classroom product                             |
|---------|---------------------------|-----------------------------------------------|
| 1       | Recommendation approaches | Design a structured class survey              |
| 2       | Similarity scoring        | Implement and test the matching function      |
| 3       | Recommendation logic      | Complete the end-to-end Python system         |
| 4       | Algorithmic influence     | Feed experiment and ethical design discussion |

### Assessment evidence

Trace similarity code, implement a valid recommendation, explain both approaches, and analyze social and ethical consequences.

# Assessment Framework

A balanced model for code, concepts, evidence, and responsibility

Assessment should capture both programming competence and AI literacy. A correct-looking output is not sufficient: students should be able to explain the algorithm, justify the data or test set, and identify limitations.

## Recommended evidence portfolio

| Evidence       | Examples                                             | What it demonstrates                                      |
|----------------|------------------------------------------------------|-----------------------------------------------------------|
| Code traces    | Conditions, loops, functions, list search            | Prediction, state tracking, and algorithmic understanding |
| Short builds   | Validator, text analyzer, similarity function        | Accurate syntax and focused problem solving               |
| Investigations | Quick, Draw!, tokenizer, model bias                  | Observation, evidence, and explanation                    |
| Projects       | Balanced Day, semantic game, classifier, recommender | Integration, iteration, and user-centered design          |
| Reflections    | Risk analysis, fairness review, deployment checklist | Responsible AI reasoning and metacognition                |

## Common four-level rubric

| Dimension        | Beginning                       | Developing                     | Secure                                          | Extending                                                   |
|------------------|---------------------------------|--------------------------------|-------------------------------------------------|-------------------------------------------------------------|
| Correctness      | Does not run or meet the task   | Partly works; major cases fail | Works for expected cases                        | Robust, tested, and handles boundary cases                  |
| Design           | Single block; unclear variables | Some decomposition             | Clear functions and data flow                   | Well-named, reusable, and economical components             |
| AI understanding | Uses terms without explanation  | Partial explanation            | Connects behavior to data/model                 | Uses evidence and states uncertainty or limits              |
| Responsibility   | Ignores risk                    | Names a generic risk           | Identifies specific stakeholders and safeguards | Tests fairness, minimizes data, and plans monitoring/appeal |

## Suggested local weighting (adaptable)

|                                            |                                   |                                               |                                        |                                                      |
|--------------------------------------------|-----------------------------------|-----------------------------------------------|----------------------------------------|------------------------------------------------------|
| <b>35%</b><br>Coding labs and short builds | <b>25%</b><br>Integrated projects | <b>15%</b><br>Concept checks and code tracing | <b>15%</b><br>Portfolio and reflection | <b>10%</b><br>Collaboration and responsible practice |
|--------------------------------------------|-----------------------------------|-----------------------------------------------|----------------------------------------|------------------------------------------------------|



# Implementation, Safety, and Inclusion

Conditions for responsible classroom use

## Technical and organizational preparation

- Python environment with approved packages and a predictable reset process.
- Organization-approved AI service and teacher-controlled API credentials.
- Access to Quick, Draw! and Teachable Machine, plus offline alternatives.
- Sample datasets, test images, printed cards, and starter-code versions.
- A classroom procedure for reporting errors, unsafe output, or access problems.

## Non-negotiable safeguards

- Never place API keys in student code, screenshots, shared documents, or repositories.
- Do not submit names, faces, health data, or other personal information without authorization.
- Treat model output as a draft to verify, not as an authority.
- Discuss copyright, attribution, licenses, and permitted reuse of datasets and outputs.
- Keep a human decision-maker and an appeal route for consequential decisions.

## Inclusive classroom design

- Offer starter code, partially completed functions, and extension challenges without changing the core learning goal.
- Use paired roles such as driver, navigator, tester, and evidence recorder, rotating roles deliberately.
- Provide non-camera alternatives for visual-classification work and offline alternatives for web activities.
- Use culturally varied examples and invite students to identify what a dataset may be missing.
- Assess explanations and testing decisions as well as typing speed and final code volume.

## Teacher readiness checklist

| When                | Check                                                                               |
|---------------------|-------------------------------------------------------------------------------------|
| Before the unit     | Verify accounts, access, key storage, packages, and offline fallback.               |
| Before each project | Define allowed data, expected output, test criteria, and stopping rules.            |
| During use          | Monitor prompts, data entry, unexpected output, and unequal performance.            |
| After use           | Review evidence, document failures, remove unnecessary data, and revise safeguards. |

# Source and Adaptation Notes

How to use this working curriculum

## Primary source

Israel Ministry of Education, Administration for Innovation and Technology, supervision of Computer Science, Artificial Intelligence, Software Engineering and Cyber Education. Artificial Intelligence Integrated with Computer Science for Middle School: AI2 with Algorithmics through Python, Part B.

Official source PDF:

<https://meyda.education.gov.il/files/CSIT/K7/ai-Studies-Program-b.pdf>

## Scope of this edition

- This document is an English implementation-oriented synthesis, not an official translation.
- Unit goals, central concepts, teaching approaches, assessment directions, and total hours are derived from the official source.
- Meeting sequences, portfolio structure, rubric, weighting, safety checklist, and some project framing are practical recommendations for local implementation.
- The official source identifies Grade 8. The sequence can be adapted for Grade 9 by increasing independence, code complexity, testing expectations, and project scope.
- Platform names in the source may be replaced with locally approved equivalents while preserving the learning objective.

## Suggested Grade 9 adaptation

### More code

Reduce starter code and require reusable modules and tests.

### More evidence

Use larger external test sets and subgroup comparisons.

### More independence

Students plan data collection, iteration, and project milestones.

### More responsibility

Add model cards, risk logs, and stakeholder review.

### The year-long message

**Students move from consumers of AI to informed builders who can code, test, question, and improve intelligent systems responsibly.**